

# VyaparFlow

## High-Level System Design Transcript

Architecture, Tech Stack, Deployment, Data Flow, Scalability, and Operational View

*This document is written as a spoken technical transcript for explaining the high-level design of VyaparFlow. It intentionally focuses on system architecture and design decisions. Low-level design, detailed schemas, class diagrams, method-level APIs, and sequence-level internals can be prepared separately in the next phase.*

|                    |                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------|
| Project            | VyaparFlow                                                                                  |
| System Type        | Voice-first Agentic Business Operating System for micro and small enterprises               |
| Primary Goal       | Convert natural business speech into completed business workflows                           |
| Architecture Style | Layered, service-oriented, event-aware, agent-orchestrated architecture                     |
| Core Workloads     | Voice processing, intent detection, agent planning, inventory, finance, analytics, insights |
| Design Priority    | Accessibility, correctness, workflow automation, observability, privacy, scalability        |

## 1. HLD Opening

### Presenter:

In this high-level system design, I will explain how VyaparFlow is structured from an architecture point of view. VyaparFlow is a voice-first Agentic Business Operating System that allows a small business owner to speak a business event, such as a sale or expense, and the system converts it into completed operations across inventory, finance, analytics, and recommendations.

### Presenter:

At a high level, the system is designed around three principles. First, user interaction should be natural and voice-first. Second, the backend should be modular, so inventory, finance, analytics, and future modules can evolve independently. Third, AI agents should not directly mutate data without controlled business services, validation, and audit logs.

## 2. Scope of High-Level Design

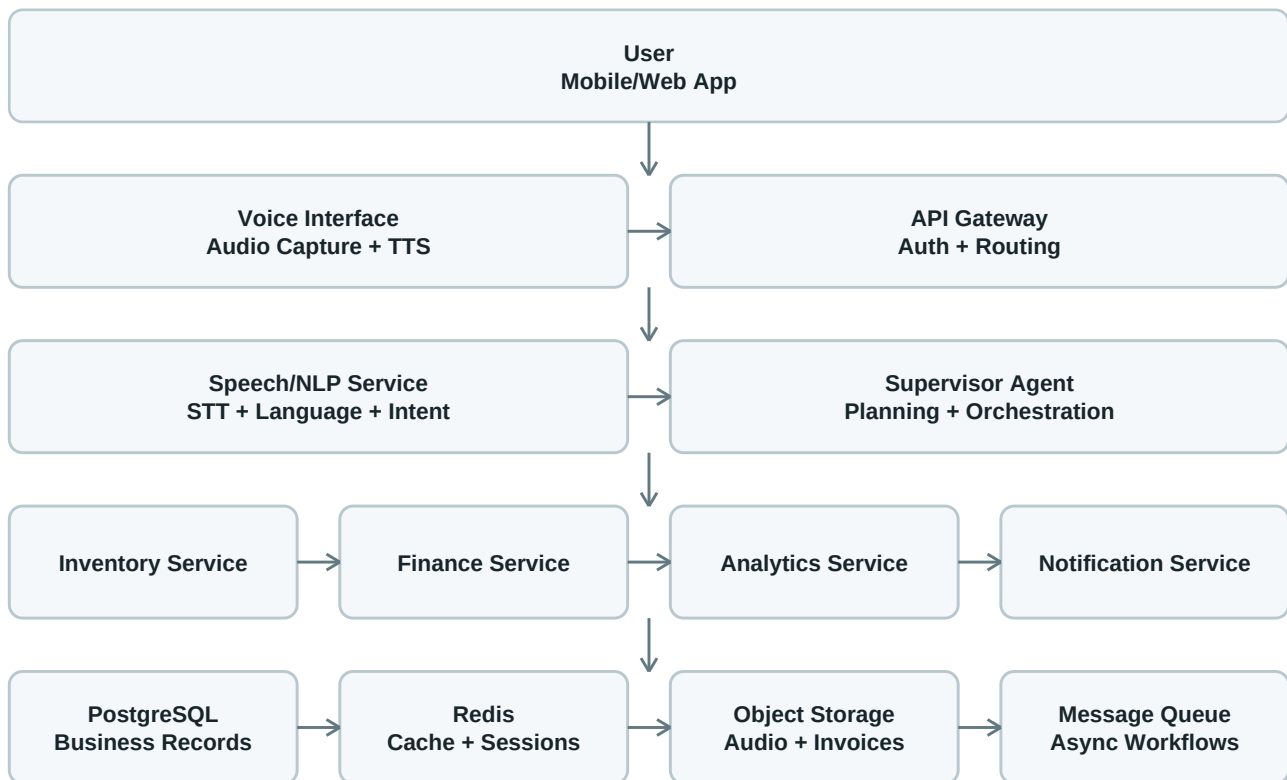
The high-level design covers the following system aspects:

- Logical architecture and major layers.
- Core services and agent responsibilities.
- Recommended technology stack.
- End-to-end request and data flow.
- Deployment topology for MVP and scalable production.
- Database, cache, storage, queue, and analytics choices.
- Security, privacy, observability, reliability, and scaling strategy.
- Key design trade-offs and future extension points.

*Low-level design will later expand this into exact classes, database tables, API contracts, sequence diagrams, validations, and pseudocode.*

## 3. Logical Architecture Diagram

## Logical Architecture



### Presenter:

This diagram shows the logical architecture. The user interacts through a mobile or responsive web application. The voice interface captures audio and can return voice responses. Requests pass through an API gateway, which handles authentication, routing, rate limiting, and request tracing. The Speech and NLP service converts speech to text, detects the language, extracts intent, and identifies business entities like product, quantity, price, customer, date, and payment status.

### Presenter:

The Supervisor Agent is the orchestration brain. It creates a workflow plan and calls domain services through controlled tools. The domain services are responsible for actual business state changes. This separation is important because AI can decide what needs to happen, but deterministic backend services should enforce business rules and consistency.

4. Architecture Layers

| Layer                     | Purpose                                                           | Major Components                                                                |
|---------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Experience Layer          | Provides simple voice-first interaction and dashboard access.     | Mobile app, responsive web app, voice recorder, dashboard, reports.             |
| Access Layer              | Protects and routes all requests.                                 | API gateway, authentication, authorization, rate limiting.                      |
| AI Processing Layer       | Understands natural speech and converts it into structured tasks. | Speech-to-text, language detection, intent/entity extraction, prompt templates. |
| Agent Orchestration Layer | Plans and coordinates workflows across modules.                   | Supervisor Agent, workflow planner, task state, retry and clarification logic.  |
| Domain Service Layer      | Executes validated business operations.                           | Inventory, Finance, Analytics, Customer, Notification, Invoice services.        |
| Data Layer                | Stores durable business data and supports fast access.            | PostgreSQL, Redis, object storage, vector store, message queue.                 |
| Operations Layer          | Keeps the platform reliable and observable.                       | Logs, metrics, traces, alerts, backups, CI/CD, monitoring.                      |

5. Core Components

Client Application

The client provides voice recording, text fallback, dashboard views, transaction history, inventory view, reports, and confirmation prompts. For MVP, a responsive web app is enough; later it can become a mobile app.

API Gateway

The gateway centralizes authentication, authorization checks, routing, request limits, versioning, and trace IDs. It keeps public client traffic separate from internal services.

Speech and NLP Service

This service transforms audio into text, detects language, normalizes the command, extracts entities, and gives the Supervisor Agent a structured request.

Supervisor Agent

The Supervisor Agent decides the workflow. It can ask clarification questions, call domain tools, combine results, and produce a final response for the user.

Domain Services

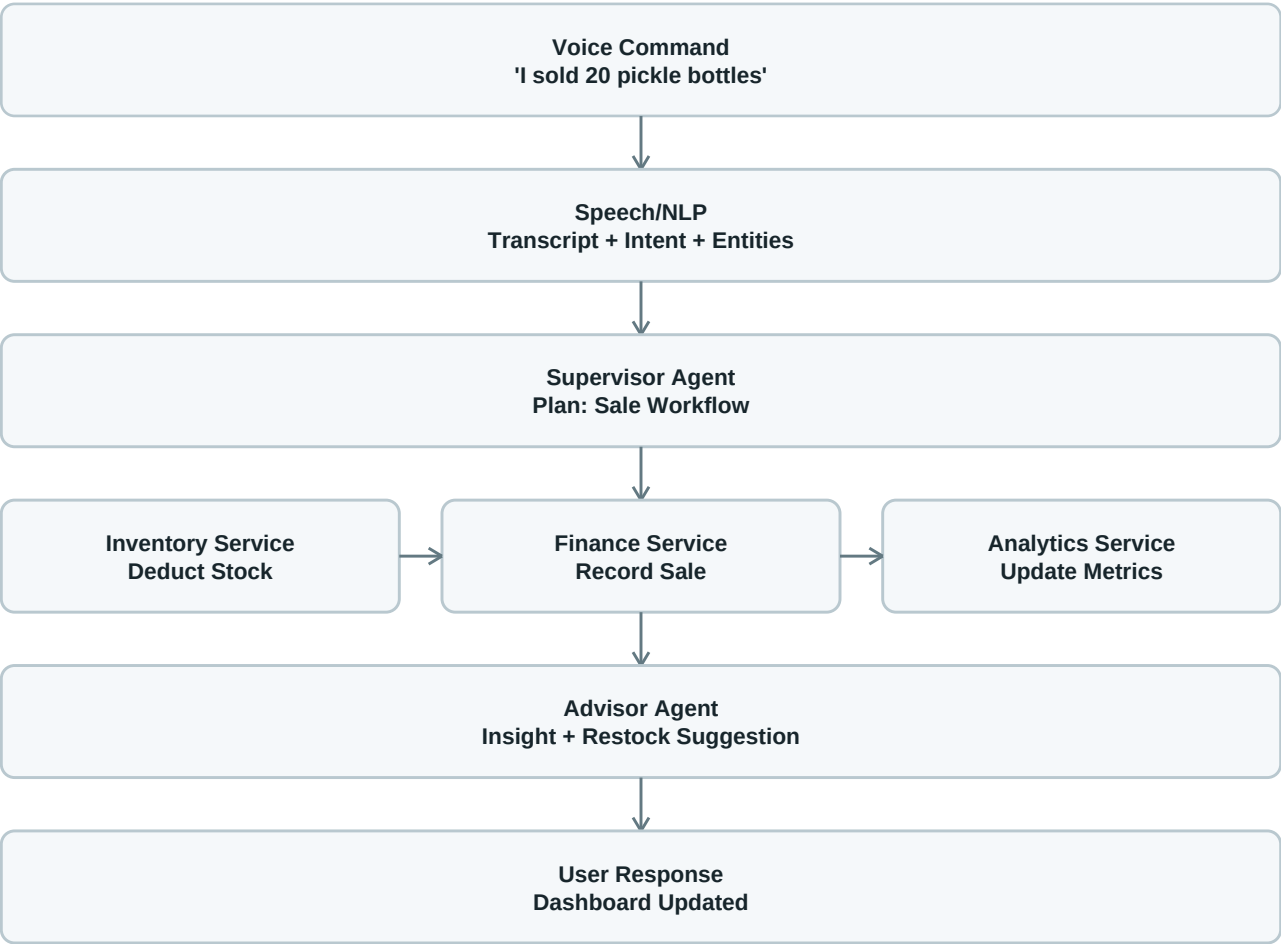
Inventory, Finance, Analytics, Customer, Invoice, and Notification services execute deterministic business rules and persist state changes.

Data Platform

The data platform includes PostgreSQL for transactional records, Redis for cache and session state, object storage for generated files and optional audio, a queue for async jobs, and analytics tables for reporting.

6. Agent Workflow Diagram

Agent-Orchestrated Business Workflow



Presenter:

The agent workflow starts with a natural command. The system extracts structured meaning from speech, then the Supervisor Agent decides that this is a sale workflow. It calls the Inventory Service to reduce stock, the Finance Service to record revenue, and the Analytics Service to update summaries. If stock falls below threshold, the Advisor Agent can recommend restocking. Finally, the user receives a confirmation and the dashboard is updated.

Important design rule:

Agents should be allowed to reason, plan, and recommend actions, but all database writes should happen through domain services with validation, transaction handling, idempotency, and audit logs.

7. Recommended Technology Stack

| Area          | Recommended Stack                                                | Reason                                                             |
|---------------|------------------------------------------------------------------|--------------------------------------------------------------------|
| Frontend      | React or Next.js, TypeScript, Tailwind CSS                       | Fast MVP development, responsive UI, strong ecosystem.             |
| Mobile Future | React Native or Flutter                                          | Single codebase for Android/iOS when mobile app becomes necessary. |
| Backend API   | Node.js with NestJS or Python FastAPI                            | Clean service structure, good API development, async support.      |
| Agent Layer   | Python orchestration service or backend-integrated agent runtime | Python has strong AI tooling; separation helps scale AI workloads. |

| Area           | Recommended Stack                                   | Reason                                                                             |
|----------------|-----------------------------------------------------|------------------------------------------------------------------------------------|
| Database       | PostgreSQL                                          | Reliable transactional storage for products, sales, expenses, customers, invoices. |
| Cache          | Redis                                               | Session state, rate limits, temporary workflow state, frequent dashboard counters. |
| Queue          | RabbitMQ, Redis Queue, or cloud queue               | Async reports, notifications, invoice generation, retries.                         |
| Object Storage | S3-compatible storage                               | Audio files, generated invoices, exports, report PDFs.                             |
| Vector Store   | pgvector or managed vector DB                       | Long-term business memory and retrieval of past context.                           |
| AI Services    | Speech-to-text, LLM, embeddings, text-to-speech     | Natural language understanding and conversational responses.                       |
| Observability  | OpenTelemetry, Prometheus/Grafana, centralized logs | Trace AI workflows and backend behavior.                                           |
| Deployment     | Docker, Kubernetes or managed containers            | Portable deployment and independent scaling.                                       |

8. Data Flow

End-to-End Data Flow



The data flow begins when the user submits audio or text. The API gateway authenticates the request and attaches a trace ID. Speech and NLP services create a structured interpretation. The Supervisor Agent builds a workflow plan with tasks and dependencies. Domain services validate and execute the operations. The database commit and audit log are completed before the user receives final confirmation.

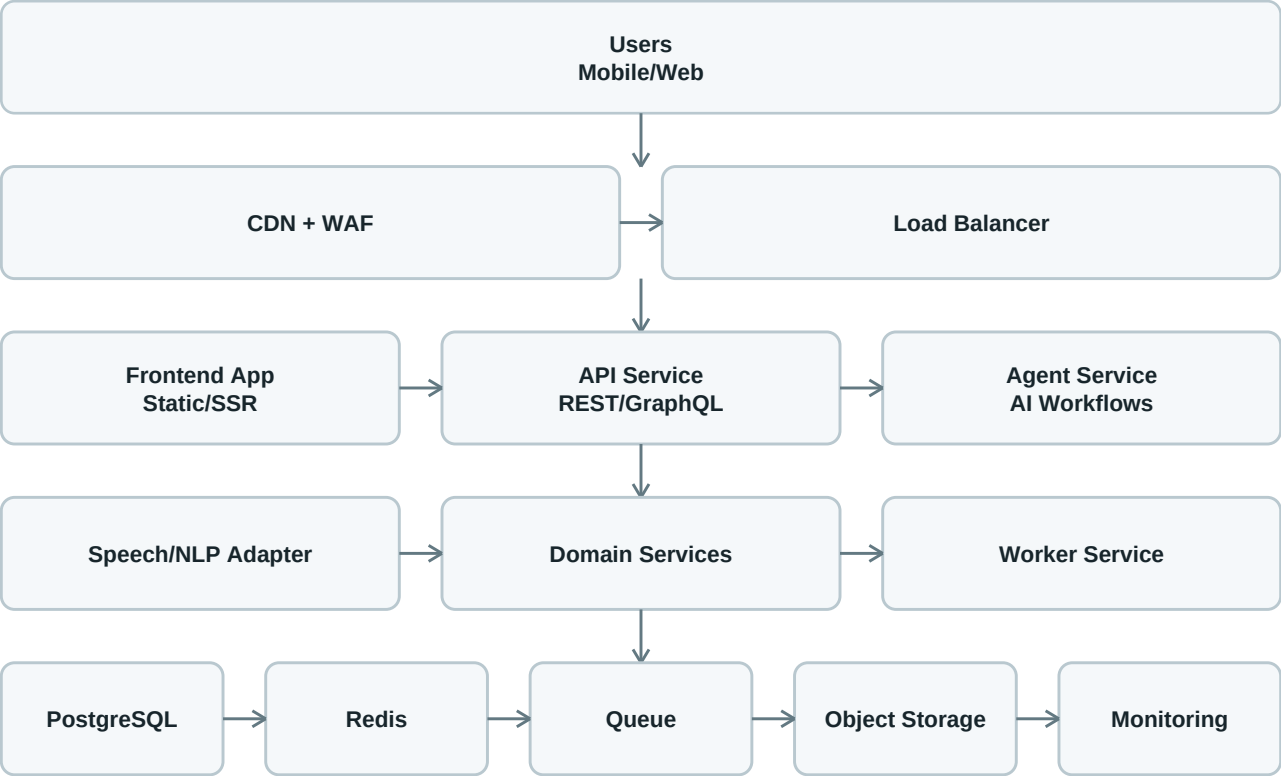
9. Data Storage Design at HLD Level

| Storage        | Used For                                                                                  | Notes                                                           |
|----------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| PostgreSQL     | Users, businesses, products, stock movements, transactions, expenses, invoices, customers | Primary source of truth with ACID transactions.                 |
| Redis          | Session data, short-lived workflow state, duplicate command checks, dashboard cache       | Improves latency and protects the database from repeated reads. |
| Object Storage | Audio recordings if retained, invoice PDFs, exports, report files                         | Use lifecycle policies and avoid storing audio unless needed.   |
| Vector Store   | Business memory, prior user preferences, retrieved knowledge                              | Use with access controls and tenant isolation.                  |

| Storage       | Used For                                        | Notes                                                  |
|---------------|-------------------------------------------------|--------------------------------------------------------|
| Queue Storage | Pending async jobs, retries, notification tasks | Decouples long-running work from user request latency. |

10. Deployment Architecture Diagram

Production Deployment Topology





**Presenter:**

For deployment, a production-ready setup can use CDN and WAF at the edge, followed by a load balancer. The frontend can be hosted as static or server-rendered pages. The API service handles normal business requests. The Agent Service handles AI planning and orchestration. Worker services process asynchronous tasks such as reports, notifications, invoice generation, and reconciliation jobs.

- MVP deployment can be a simpler Docker Compose setup with frontend, backend, PostgreSQL, Redis, and worker.
- Production deployment should separate API, agent, worker, and database workloads for independent scaling.
- AI and speech services may be external managed services or internal services depending on cost, latency, and privacy needs.
- All services should emit logs, metrics, and traces with a shared correlation ID.

## 11. Scalability Strategy

- Scale stateless frontend and API services horizontally behind a load balancer.
- Scale agent workers separately because AI requests may be slower and more expensive than normal API calls.
- Use queue-based processing for reports, reminders, invoice generation, OCR, and future reconciliation tasks.
- Cache dashboard summaries and frequently accessed product data in Redis.
- Use database indexes on `business_id`, `product_id`, transaction date, payment status, and customer identifiers.
- Partition or archive old transaction data when businesses grow beyond MVP scale.
- Apply rate limits and cost budgets for speech, LLM, and embedding calls.

## 12. Reliability and Fault Tolerance

- Use idempotency keys for voice commands so repeated submissions do not create duplicate sales.
- Wrap inventory and finance updates in database transactions when they belong to one business event.
- Retry temporary failures through the queue with bounded retry limits.
- Ask the user for clarification when speech confidence, product match confidence, or price confidence is low.
- Use graceful degradation: if analytics is delayed, still record the sale and update analytics asynchronously.
- Back up PostgreSQL regularly and test restore procedures.
- Maintain audit logs for agent decisions, service calls, and business-critical changes.

## 13. Security and Privacy Architecture

- Authenticate users through OTP, password, or secure identity provider.
- Enforce tenant isolation using `business_id` checks in every query and service method.
- Use role-based access for owners, staff, mentors, and consultants.
- Encrypt traffic with HTTPS and protect secrets using a secret manager.
- Avoid storing raw audio by default; if audio is stored, use explicit consent and retention policies.
- Mask sensitive fields in logs and never log full payment or personal identifiers unnecessarily.
- Require confirmation for risky or irreversible actions such as deleting records or sending customer-facing documents.
- Maintain audit trails for agent-generated changes to support trust and debugging.

14. Observability and Monitoring

| Signal              | What to Track                                                                         |
|---------------------|---------------------------------------------------------------------------------------|
| API Metrics         | Request latency, error rate, throughput, failed authentication, rate-limit hits.      |
| Agent Metrics       | Workflow success rate, clarification rate, tool-call failures, average planning time. |
| Speech/NLP Metrics  | Transcription latency, confidence score, language detection accuracy, failed parsing. |
| Business Metrics    | Commands completed, sales recorded, inventory updates, report generation count.       |
| Cost Metrics        | Speech minutes, LLM tokens, embeddings, storage, queued jobs.                         |
| Reliability Metrics | Queue depth, retry count, database connection health, worker failures.                |

Presenter:

Observability is especially important because AI workflows are multi-step. A single user command may trigger speech processing, agent planning, inventory updates, financial entries, analytics, and notifications. Trace IDs should connect all of these steps so debugging is possible.

15. MVP Deployment vs Future Production

| Aspect     | MVP                                              | Future Production                                     |
|------------|--------------------------------------------------|-------------------------------------------------------|
| Frontend   | Responsive web app                               | Mobile app plus web dashboard                         |
| Backend    | Single modular API service                       | Separated API, agent, worker, analytics services      |
| Database   | Single PostgreSQL instance                       | Managed PostgreSQL with replicas, backups, PITR       |
| AI Layer   | Managed speech and LLM APIs                      | Hybrid managed/internal services with cost controls   |
| Queue      | Redis queue or simple background worker          | Dedicated queue with retries and dead-letter handling |
| Deployment | Docker Compose or one managed container platform | Kubernetes or managed container services              |
| Monitoring | Basic logs and uptime checks                     | Metrics, traces, alerts, dashboards, cost monitoring  |

16. Key Design Decisions

- Use a modular service architecture instead of a single AI-heavy monolith.
- Keep the Supervisor Agent responsible for planning, while domain services remain responsible for validation and persistence.
- Use PostgreSQL as the source of truth because business records need transactional consistency.
- Use queues for slow or non-critical work so voice commands remain responsive.
- Keep audio retention optional to reduce privacy risk and storage cost.
- Design future modules as additional agents and services rather than rewriting the core platform.

17. Risks and Mitigations

| Risk                        | Mitigation                                                                               |
|-----------------------------|------------------------------------------------------------------------------------------|
| Wrong speech interpretation | Use confidence thresholds, confirmations, language support, and correction flows.        |
| Duplicate transactions      | Use idempotency keys, command history, and duplicate detection windows.                  |
| Agent hallucination         | Restrict agents to approved tools, validate service inputs, and audit all writes.        |
| Slow AI responses           | Use streaming status, async processing, caching, and model fallback strategies.          |
| Data privacy issues         | Minimize audio storage, encrypt data, mask logs, and enforce tenant isolation.           |
| Cost growth                 | Track AI usage, cache results, set rate limits, and use cheaper models for simple tasks. |
| Scaling database writes     | Use indexes, optimized queries, archiving, and read replicas for analytics.              |

18. Closing HLD Summary

Presenter:

To summarize the high-level design, VyaparFlow is built as a layered, service-oriented, agent-orchestrated platform. The user interacts naturally through voice. Speech and NLP convert that voice into structured business intent. The Supervisor Agent plans the workflow, while deterministic domain services execute inventory, finance, analytics, and notification operations. PostgreSQL stores the source of truth, Redis and queues improve performance, and object storage handles generated files and optional audio.

Presenter:

This design keeps the system accessible for small business owners while still being technically scalable. It supports the MVP today and leaves clear extension points for future agents such as marketing, customer support, GST assistance, OCR bills, WhatsApp integration, and UPI reconciliation.

*HLD takeaway: VyaparFlow uses voice as the interface, agents as the workflow brain, domain services as the rule enforcers, and reliable data systems as the business source of truth.*

Recommended presentation duration: 10 to 15 minutes for full HLD discussion. For a short version, cover sections 3, 6, 7, 10, 11, and 18.